Conexão com Banco de Dados

API JDBC

Utilizando o XAMPP

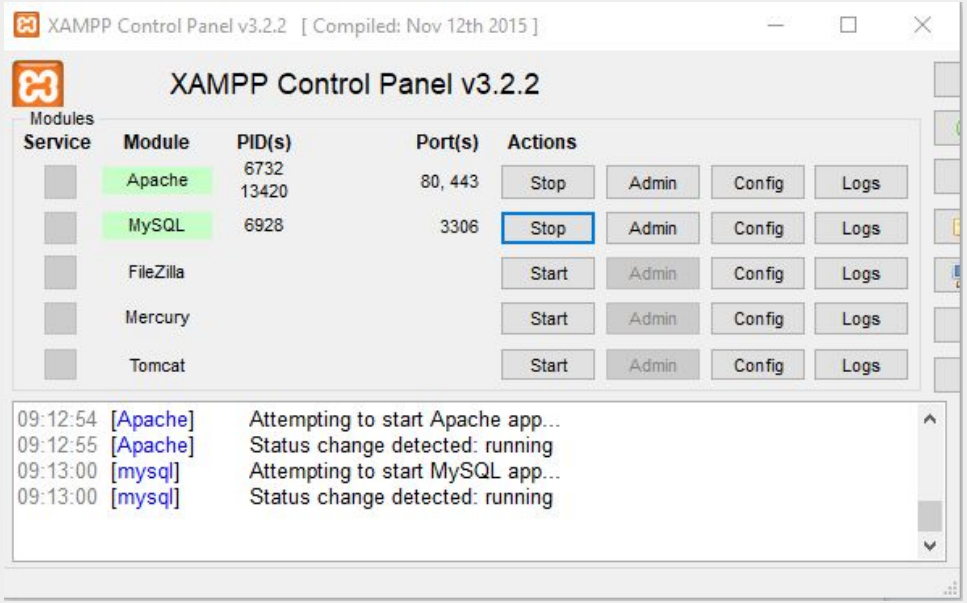
- É um servidor independente de plataforma e software livre;
- Consiste principalmente na base de dados MariaDB, no servidor web Apache e nos interpretadores para linguagens PHP e Perl;
- Fácil instalação.



Download: https://www.apachefriends.org/download.html

Executando o XAMPP

Para acessar o banco de dados MySQL, você precisará iniciar o serviço do próprio MySQL e do Apache.
Ativados os dois serviços, clique em Admin do MySQL.
Com isso você acessará o phpMyAdmin.



phpMyAdmin

É uma ferramenta que auxilia na administração do banco de dados.

Proporciona uma interface amigável e de fácil manipulação.



Banco de Dados - MariaDB (compatível com MySQL)

Utilizaremos o MariaDB (que acompanha o XAMPP).

- SGBD de código aberto, compatível com o MySQL.
- Utiliza SQL (Structured Query Language).
- Popular pela vasta aplicação em projetos;
- Gratuito;
- Fácil utilização;

Banco de dados relacional

Um banco de dados relacional armazena dados em tabelas. Tabelas são organizadas em colunas, e cada coluna armazena um tipo de dados (inteiro, números reais, strings de caracteres, data, etc).

Banco de dados



Tabela

1	**id**	int(11)
2	**nome**	varchar(40)
3	**cargo**	varchar(40)
4	**salario**	decimal(7,2)

Banco de dados relacional - Tabelas

Estrutura da Tabela

1	**id** 🔑	int(11)
2	**nome**	varchar(40)
3	**cargo**	varchar(40)
4	**salario**	decimal(7,2)

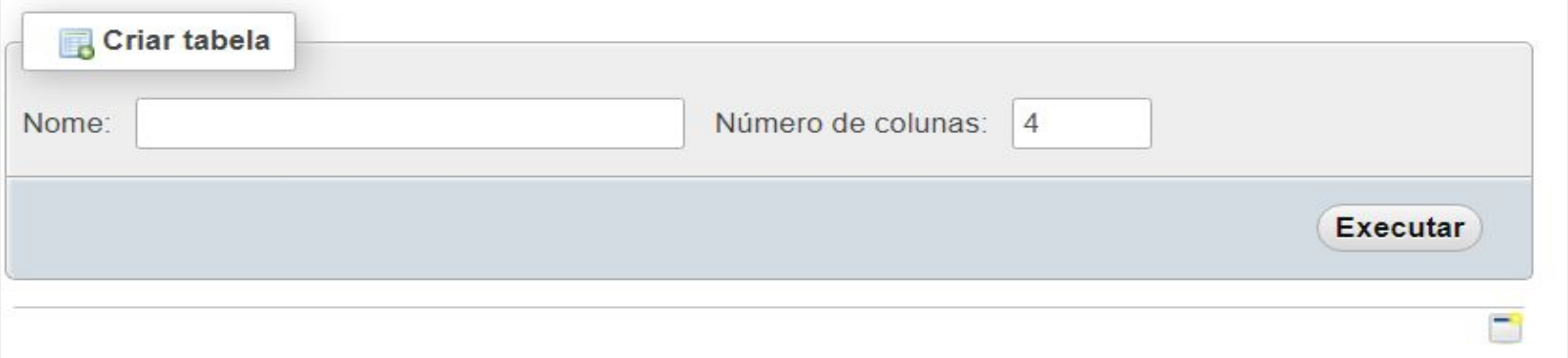
Os dados de uma simples “instância” de uma tabela são armazenados como uma linha. Por exemplo, a tabela funcionarios tem as colunas como id, nome, cargo, salario. Uma linha na tabela é {1, Mel, Programador Pleno I, 7345.44}.

Banco de dados relacional - Tabelas

Nomeie a base de dados e clique criar.



Nomeie a tabela e escolha a quantidade de colunas. Para esse exemplo, foi escolhido 4.



Banco de dados relacional - Tabelas

A imagem apresenta os campos da tabela funcionarios.

Nome	Tipo	Tamanho/Valores*	Predefinido	Agrupamento (Collation)	Atributos	Nulo	A_I
id Escolher a partir das Colunas Centrais	INT	11	Nenhum			☐	☑
nome Escolher a partir das Colunas Centrais	VARCHAR	40	Nenhum	utf8mb4_unicode_		☐	☐
cargo Escolher a partir das Colunas Centrais	VARCHAR	40	Nenhum	utf8mb4_unicode_		☐	☐
salario Escolher a partir das Colunas	DECIMAL	7,2	Nenhum			☐	☐

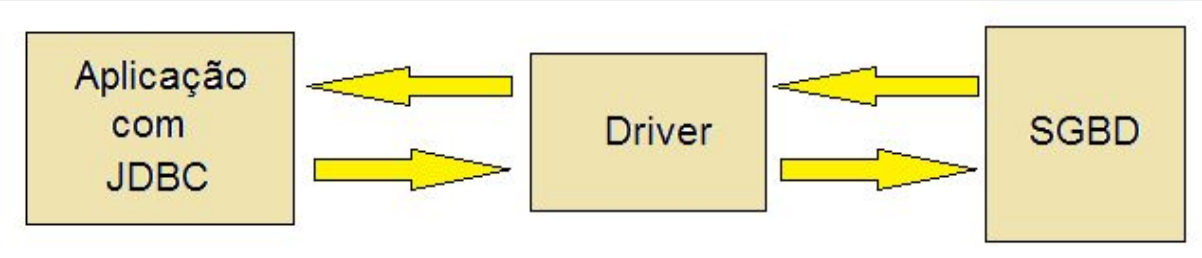
Banco de dados relacional - Tabelas

Selecione o id como auto incremento. Automaticamente esse campo será a chave primária da tabela. Clique em executar e em seguida clique em Guarda (Canto inferior direito).

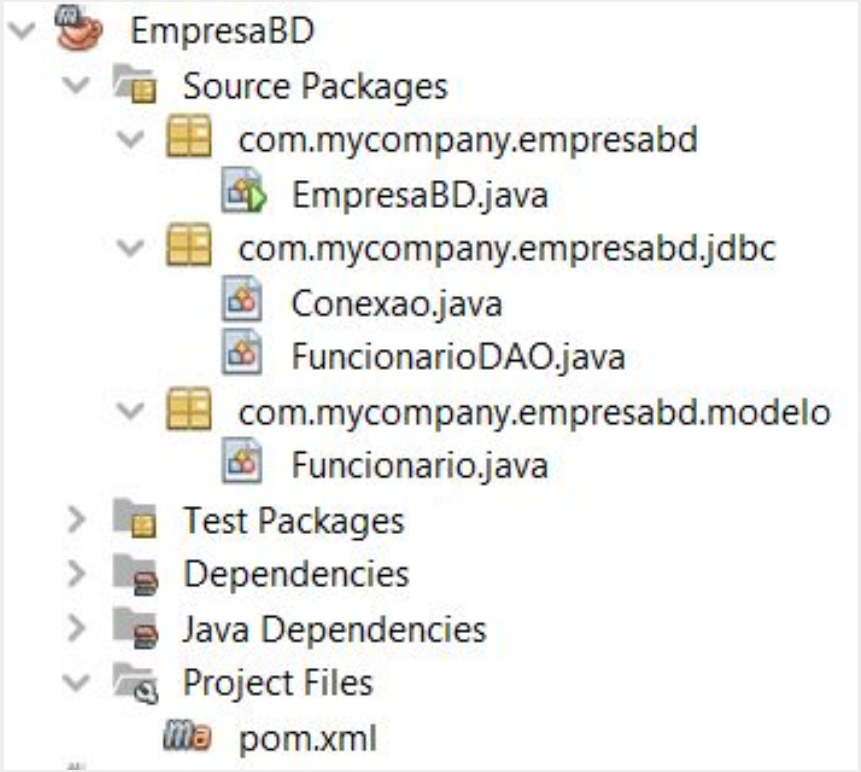
#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra
1	**id** 🔑	int(11)			Não	*Nenhum*		AUTO_INCREMENT
2	**nome**	varchar(40)	utf8mb4_unicode_ci		Não	*Nenhum*		
3	**cargo**	varchar(40)	utf8mb4_unicode_ci		Não	*Nenhum*		
4	**salario**	decimal(7,2)			Não	*Nenhum*		

API JDBC

- Interface de Programação de Aplicativos JDBC (**J**ava **D**ata**B**ase **C**onnectivity);
- É uma tecnologia que permite a conexão com Sistemas Gerenciadores de Banco de Dados;
- Foco na aplicação;
- A complexidade da conexão com o banco de dados fica na responsabilidade do JDBC.
- A aplicação sofrerá mínimas mudanças, caso alteremos a base de dados.



Projeto Java - Estrutura do projeto



Projeto Java - O arquivo pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.mycompany</groupId>
    <artifactId>EmpresaBD</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>18</maven.compiler.source>
        <maven.compiler.target>18</maven.compiler.target>
        <exec.mainClass>com.mycompany.empresabd.EmpresaBD</exec.mainClass>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.mariadb.jdbc</groupId>
            <artifactId>mariadb-java-client</artifactId>
            <version>3.5.1</version>
        </dependency>
    </dependencies>
</project>
```

Projeto Java - A classe Conexao

```java
package com.mycompany.empresabd.jdbc;

import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Connection;

/*
 *  Professor Gerson Risso
 */
public class Conexao {

    private static final String URL = "jdbc:mariadb://localhost:3306/empresa";
    private static final String USUARIO = "root";
    private static final String SENHA = "";
    private static Conexao con;
    private Connection connection;

    private Conexao() throws SQLException {
        connection = DriverManager.getConnection(URL, USUARIO, SENHA);
    }

    public static Conexao getConexao() throws SQLException {
        if (con == null) {
            con = new Conexao();
        }
        return con;
    }
```

Projeto Java - A classe Conexao

```
    public Connection getConnection() {
        return connection;
    }

    public void fechar() throws SQLException {
        if (connection != null && !connection.isClosed()) {
            connection.close();
        }
        con = null;
    }
}
```

Projeto Java - A classe Funcionario

```java
package com.mycompany.empresabd.modelo;

import java.math.BigDecimal;

/*
 *  Professor Gerson Risso
 */
public class Funcionario {
  private int id;
    private String nome;
    private String cargo;
    private BigDecimal salario;

    public Funcionario() {
    }

    public Funcionario(String nome, String cargo, BigDecimal salario) {
        this.nome = nome;
        this.cargo = cargo;
        this.salario = salario;
    }
```

Projeto Java - A classe Funcionario

```
public int getId() {...3 lines }

public void setId(int id) {...3 lines }

public String getNome() {...3 lines }

public void setNome(String nome) {...3 lines }

public String getCargo() {...3 lines }

public void setCargo(String cargo) {...3 lines }

public BigDecimal getSalario() {
    return salario;
}

public void setSalario(BigDecimal salario) {
    this.salario = salario;
}

@Override
public String toString() {...4 lines }
```

Projeto Java - A classe FuncionarioDAO

```java
package com.mycompany.empresabd.jdbc;

import com.mycompany.empresabd.modelo.Funcionario;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/*
 *  Professor Gerson Risso
 */
public class FuncionarioDAO {

    private Connection connection;
    private PreparedStatement ps;
    private ResultSet rs;
```

Projeto Java - A classe FuncionarioDAO

```java
public List<Funcionario> listarFuncionarios(Conexao conexao) throws SQLException {
    try {
        List<Funcionario> lista = new ArrayList<>();
        String sql = "SELECT id, nome, cargo, salario FROM funcionarios";
        connection = conexao.getConnection();
        ps = connection.prepareStatement(sql);
        rs = ps.executeQuery();
        while (rs.next()) {
            Funcionario f = new Funcionario();
            f.setId(rs.getInt("id"));
            f.setNome(rs.getString("nome"));
            f.setCargo(rs.getString("cargo"));
            f.setSalario(rs.getBigDecimal("salario"));
            lista.add(f);
        }
        return lista;
```

Projeto Java - A classe FuncionarioDAO

```
            } finally {
                if (rs != null) {
                    rs.close();
                }
                if (ps != null) {
                    ps.close();
                }
            }
        }
    }
```

Projeto Java - A classe EmpresaBD

```
package com.mycompany.empresabd;
import com.mycompany.empresabd.jdbc.Conexao;
import com.mycompany.empresabd.modelo.Funcionario;
import com.mycompany.empresabd.jdbc.FuncionarioDAO;
import java.sql.SQLException;
import java.util.List;

public class EmpresaBD {

    public static void main(String[] args) throws InterruptedException {
        Conexao conexao = null;
        try {
            conexao = Conexao.getConexao();
            System.out.println("Conectado!");
            FuncionarioDAO dao=new FuncionarioDAO();
            List<Funcionario>lista=dao.listarFuncionarios(conexao);
            for(Funcionario f:lista){
                System.out.println(f);
            }
        } catch (SQLException e) {
            System.out.println("Erro ao conectar: " + e.getMessage());
```

Projeto Java - A classe EmpresaBD

```
        } finally {
            try {
                if (conexao != null) {
                    conexao.fechar();
                    System.out.println("Encerrado!");
                }
            } catch (SQLException e) {
                System.out.println("Falha ao fechar: " + e.getMessage());
            }
        }
    }
}
```